

Operating systems

Oscar Palm

SP 1 2025

Contents

1	Introduction	4
1.1	Introduction to C	4
1.2	Course introduction	4
1.2.1	Why study OS?	4
1.3	What is an Operating system	5
1.4	The event-driven (interrupt-drive) life of an OS	5
1.5	System calls (overview)	6
1.6	Dual-Mode and Multimode Operation	6
1.7	Operating system services	6
1.8	System calls (continued) and APIs [Self reading]	6
1.9	System boot [Self reading]	6
1.10	Operating systems structures	6
2	Processes	7
2.1	Introduction to processes	7
2.1.1	Context switching	8
2.2	Process scheduling	9
2.3	Operations on processes	10
2.3.1	Process creation	10
2.3.2	Process termination	10
2.4	Interprocess communication	11
3	Multithread Programming	12

3.1	Threads (Introduction)	12
3.1.1	Concurrency vs Parallelism	12
3.2	Multithreading models	12
3.2.1	User Threads and Kernel Threads	12
3.2.2	Many-to-One	13
3.2.3	One-to-one	13
3.2.4	Many-to-many	13
3.3	Implicit threading	13
3.4	Threading issues	13
4	Process scheduling	14
4.1	Introduction to process scheduling	14
4.1.1	CPU-bound	14
4.1.2	I/O bound	14
4.2	Batch / Interactive / Rts scheduling	14
4.2.1	Batch	14
4.2.2	Interactive	15
4.2.3	Real-time	15
4.3	Processes vs. Threads scheduling	15
4.4	Multiprocessor hardware	15
4.5	Multiprocessor scheduling	15
5	Synchronization	16
5.1	Motivation	16
5.2	The Critical Section problem	16
5.2.1	Check-competition approach	17
5.2.2	The check-order / after-you approach	17
5.2.3	Peterson's Algorithm	18
5.3	Synchronization and hardware support	18
5.3.1	Read-Modify-Write instructions	18
5.3.2	Reflection on busy-waiting	19

5.4	Semaphores	19
5.5	Other techniques	19

Lecture 1

Introduction

1.1 Introduction to C

Code snippet 1.0

```
1  #include <stdio.h> // Tells the preprocessor to load a file
2
3  int main()         // The entrypoint of the program
4  {                 // Defines a block of code, everything created in it and not returned is only defined for the block
5      printf("Hello world\n"); // A basic statement
6      return 0;
7  }
```

For this course we will use Chalmers remote StuDat computers.

1.2 Course introduction

The course book is "Modern Operating Systems", by Tanenbaum and Bos.

Vincenzo should work on his standup comedy.

1.2.1 Why study OS?

Provides services to system users. Studying it gives us an overview of how computers work on a lower level.

The operating system is both there to help the user actually use the computer, but also to protect the hardware from the main source of complications; the user. It makes sure that everything that needs to run on the computer is able to do so, and is given sufficient resources to perform its task.

1.3 What is an Operating system

An operating system is a program which provides a *set of services* to system users (human users and computer programs). It also protects the user from the hardware and vice versa.

An operating system manages resources on the computer and makes sure that all system users get the necessary resources (if possible), such as cpu(s), memory to store data, i/o devices and so on. It controls the execution of different programs, makes sure that an error in one doesn't cascade into others and makes sure different programs stay independent and cannot access each others' resources.

Saving a file

When the user is saving a file, without an operating system, nothing else would be able to happen on the computer while waiting for that; with a good operating system, the hardware resources can be managed in such a way that other tasks can be performed while waiting for the file to be written.

1.4 The event-driven (interrupt-drive) life of an OS

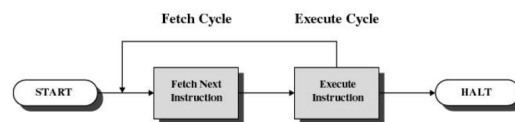


Figure 1.1: A simple fetch cycle where only one thing happens

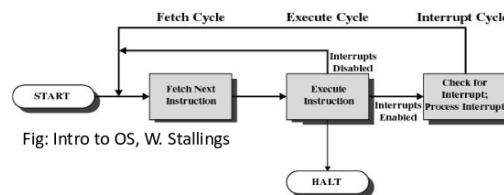


Figure 1.2: A more advanced fetch cycle where things can happen in the background

By looking for interrupts between running the instructions we are able to handle events from different parts of the system. With this we can achieve what was talked about in the example above, where other programs execute while the saving of a file was happening.

Scheduling is about choosing for how long a certain program can run before it needs to be interrupted for another program to be able to run. It is the operating system's job to choose for how long, and to choose the order the processes gets to run.

```

OS doing some work
Interrupt!
What's happening?
Key stroke! Need to print character on screen...
Doing that...
Going back to my previous work...
Interrupt!
What's happening?
Byte copied to disk!
OK then copying another byte...
Interrupt!
...

```

Figure 1.3: An example of a basic event cycle.

1.5 System calls (overview)

System calls are ways for user programs to *"talk to"* the hardware. All actions a user wants to do which modifies the hardware requires a system calls, e.g. Creating files, sending data over internet, showing something on a screen; are created through system calls. All system calls are types of interrupts. When the CPU finds the interrupt, control is given to the operating system, which chooses how to do the thing the user wants to do (if possible and the user should be able to).

1.6 Dual-Mode and Multimode Operation

What happens if a user tries to bypass the operating system by directly modifying the hardware without using system calls? The hardware keeps track of what kind of program is running, when the operating system is running, a bit is set which allows it to run "special functions" that normal users cannot.

To make sure the user cannot set that bit for themselves, when the hardware first starts, it loads the operating system into memory; during this phase the hardware and operating system defines which operations are privileged, and what code to run when those actions are performed. When the user tries to do those privileged actions, for example modifying a file, the operating system starts running.

1.7 Operating system services

1.8 System calls (continued) and APIs [Self reading]

1.9 System boot [Self reading]

1.10 Operating systems structures

Lecture 2

Processes

We want to be able to run several programs at the same time, but our cpu is only able to run one program at the time. This way of looking at the problem is impossible to solve. What we actually want is the feeling of several programs running at the same time, and while a cpu core cannot execute several programs at once, it can continually switch between executing several different processes without completing them. This is called context switching.

Think of the cpu as a human performing a task, if we for example are cooking and while cutting the vegetables we accidentally cut ourselves, we don't need to finish cooking before rinsing and plastering the wound, we instead stop cooking for a while to fix the higher priority task; stopping the bleeding, before continuing to cook.

2.1 Introduction to processes

A process is an execution of a program, the execution must be sequential. The program is the executable entity on disk, a process is when it runs, active. When the executable program is loaded into memory, it becomes a process. When a program is run several times, simultaneously or at different time, they are separate processes and cannot affect each others. A process can have sub processes.

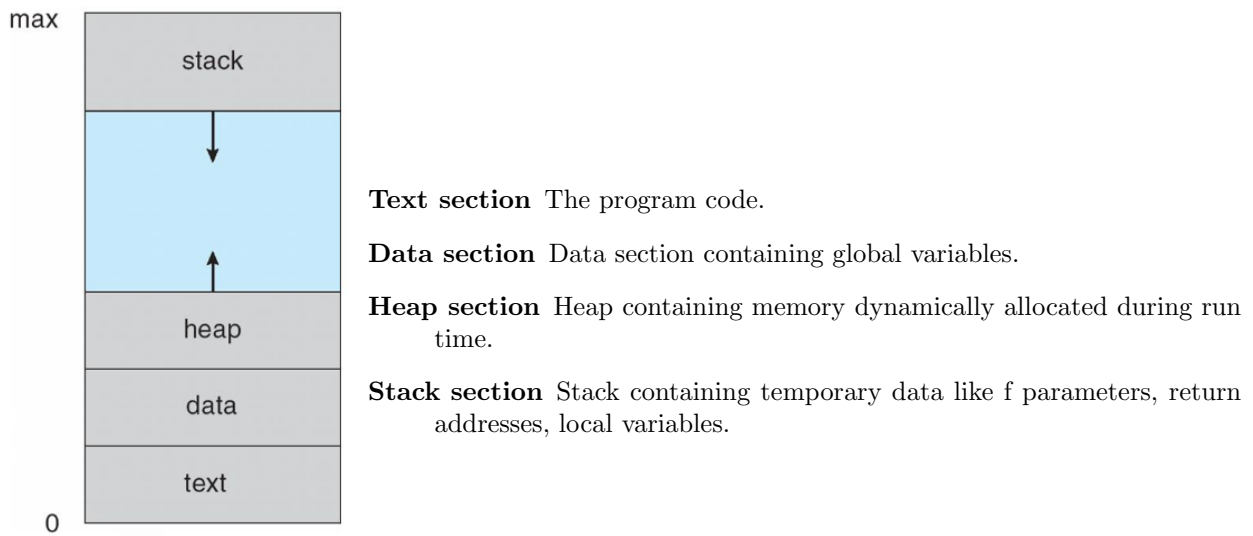


Figure 2.1: How a process' memory is allocated

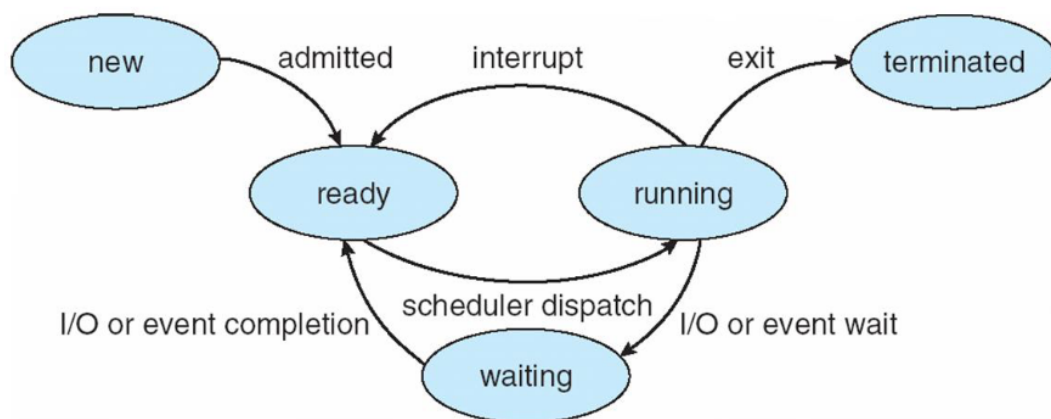


Figure 2.2: Diagram of process state

2.1.1 Context switching

To be able to switch between executing several different processes, a concept called PCB (Process Control Block) is utilized.

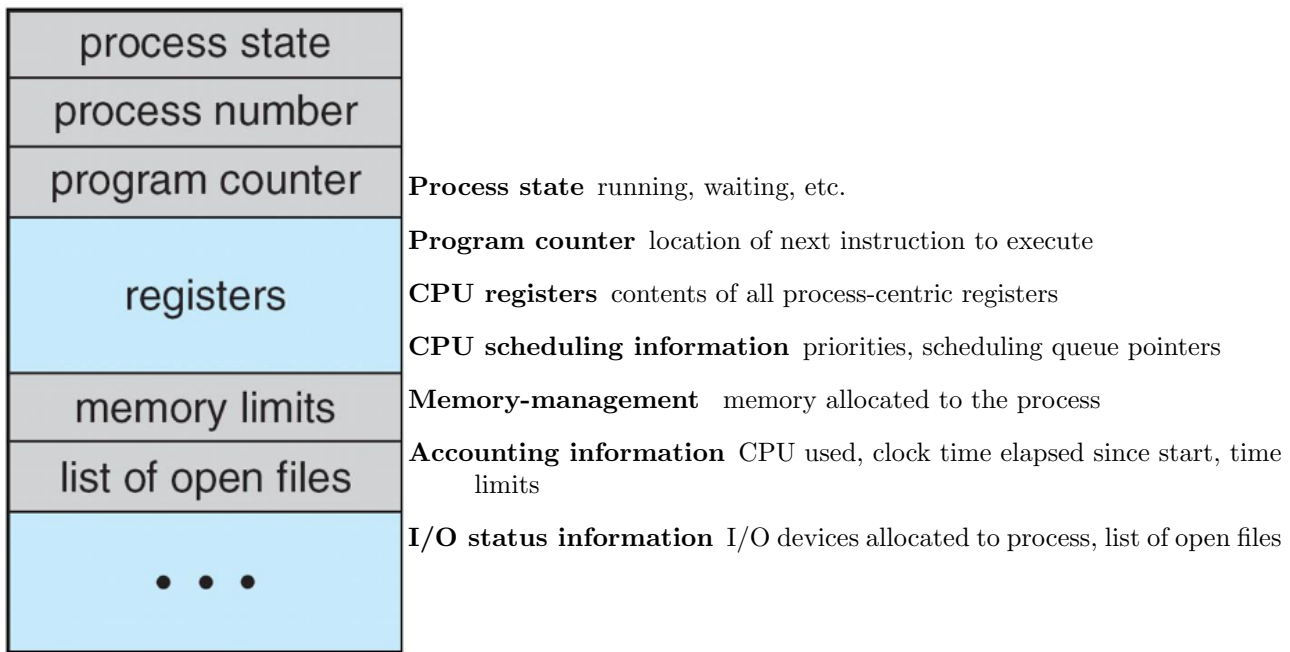


Figure 2.3: A process content block

2.2 Process scheduling

How we schedule which process should be executed next is a very important task, since a lot of processes are usually waiting for data from devices, it can be very time inefficient to store all processes in a list and just executing the first ready process. To speed up the execution we instead separate processes into different lists, when a process is waiting for input from a certain I/O device, we put it in a list connected to that I/O device, and when a process is waiting for the disk, we put it into a list connected to the disk, this way we limit the numbers of processes we need to check when choosing which process should be the next to run.

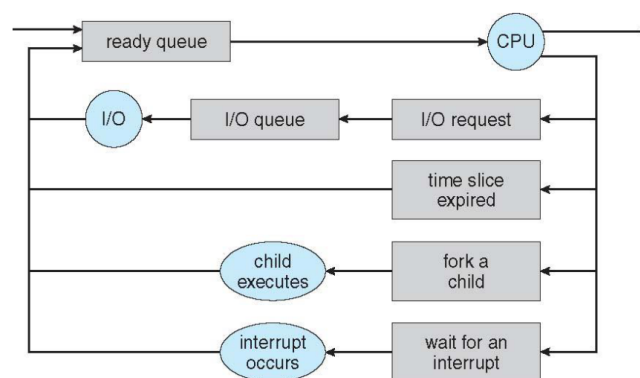


Figure 2.4: Example of how we choose next process to run

2.3 Operations on processes

2.3.1 Process creation

The user needs to be provided the ability to create and terminate processes. Since processes can't be created from nowhere, we have a higher process which can create child processes, to be able to do this, we need a system call to create processes.

From this we get a tree structure of processes, with the root process, init, always at the top, with `pid = 1`.

The command we use to be able to create new processes is `fork()`, which creates a copy of the current process, if we then want that new process to do something different than the parent program, we have a command `exec()`, which helps us load a new program into the process. NOTE: When calling `fork()`, the child and the parent will get differing results, provided nothing went wrong, the child process will get the value 0 returned while the parent gets the process ID of the created child returned.

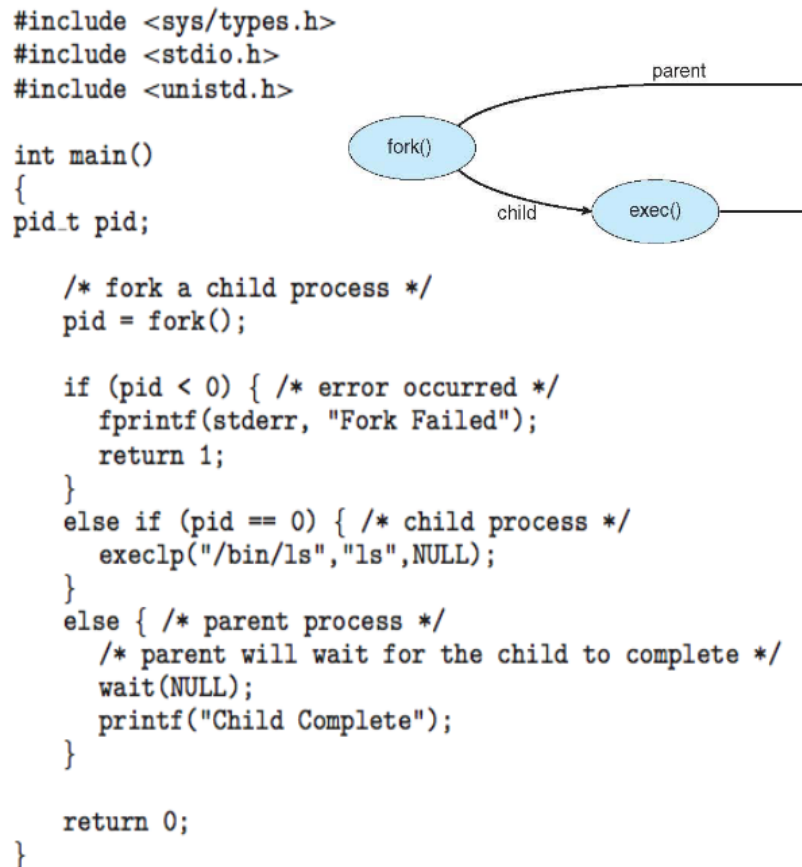


Figure 2.5: A C program creating a child

2.3.2 Process termination

When a process is finished executing, it calls the operating system using `exit()`, this tells the operating system to clean up after the program, free allocated memory and stuff.

Sometimes the termination of a process cannot be completed, for example if the child process is done, trying to return its result to the parent **but** the parent hasn't waited for the response, the process cannot be completely removed.

2.4 Interprocess communication

Lecture 3

Multithread Programming

3.1 Threads (Introduction)

A computer with multiple cores is like a body with multiple brains; at the same time, multiple instructions can be executed by the cpu. These instruction can be from different processes, but we also want to be able to run different parts of the same process on several cores.

When we talk about multithreading in programming, we talk about the operating system supporting having several program counters within the same program, this means we can execute several instructions in the same program at the same time (if the operating system have more than one core), and / or execute the program in a non-linear fashion.

One of the main reasons for creating threads within a program instead of creating new processes is that creating threads is a much simpler action for the operating systems, it takes significantly less time and resources.

3.1.1 Concurrency vs Parallelism

When we talk about executing code simultaneously we usually talk about either concurrent or parallel programs, the difference is that while a parallel program is running on different **cores** and are executing at the same time, a concurrent program is one that runs in what feels like the same time, they could execute on different cores, or they could be running on the same core, meaning they will **maybe** not execute at the same time but given enough time, both will progress.

3.2 Multithreading models

3.2.1 User Threads and Kernel Threads

User threads are threads managed by the user, the operating system is not aware of these. Think Java threads or similar.

Kernel threads are threads supported by the kernel.

Kernel threads are useful e.g. when we have a program where some parts of the program which are taking a lot of time without really blocking other things we want to do, think of a process wanting to save a file, we still want to

be able to continue running the process.

User threads are useful when we have multiple threads but want more control over when they should execute.

There are several different models for how we combine user threads and kernel threads.

3.2.2 Many-to-One

Many user threads running in one kernel thread.

3.2.3 One-to-one

One user thread running in one kernel thread.

3.2.4 Many-to-many

Allowing several user threads to be mapped to several kernel threads, look at ThreadFiber for Windows.

3.3 Implicit threading

Instead of the programmer managing threads explicitly, we can use pre-made constructions such as thread pools to speed up the assignment of threads to tasks and minimize the human errors of explicit thread management.

3.4 Threading issues

Lecture 4

Process scheduling

4.1 Introduction to process scheduling

There's two different kinds of processes; CPU-bound and I/O bound.

4.1.1 CPU-bound

A cpu bound thread is a thread that is mainly using computing and not I/O devices, the main throttle is the speed of the processor not the access time of devices.

4.1.2 I/O bound

I/O bound threads are threads which perform a lot of I/O actions and therefore have to wait a lot inbetween when it can actually compute.

4.2 Batch / Interactive / Rts scheduling

4.2.1 Batch

"business-world" applications, data analysis. Appropriate for non-preemptive.

The most important metric is throughput.

Comparison of different algorithms

Algorithm	+	-
First-Come/First-Serve (non-preemptive)	Easy, fair	Possibly inefficient (esp. for I/O bound pr
Shortest Job First (non-preemptive)	Optimal for turnaround	Starvation + need to know runtime
Shortest Remaining Time Next (preemptive)	New short jobs get good service	Starvation + need to know runtime

4.2.2 Interactive

It is not known in advance what the system is going to do. Many users that need responsiveness, requiring preemptive scheduling.

Most important metric is response time.

Comparison of different algorithms

Algorithm	+	-
Round-robin	Easy to implement	Easy to get too long or short times per process.
Priority scheduling	Can maximize the throughput easily	Harder to implement
Lottery scheduling	High control over resource use	

4.2.3 Real-time

We will not discuss this during the course, but if these do not run instantly, there is no point in running it. Think car control systems.

for short-lived, short-cycle processes with hard/soft deadlines.

Most important metric is meeting deadlines.

4.3 Processes vs. Threads scheduling

4.4 Multiprocessor hardware

4.5 Multiprocessor scheduling

Lecture 5

Synchronization

5.1 Motivation

If we run different threads in a program and they execute in parallel, this means that they might try to access data at the same time. What happens if Thread 1 tries to read variable A at the same time as Thread 2 tries to write to it? we can't know which one will access the data first.

5.2 The Critical Section problem

When writing synchronized code we create a "critical section" in which only one thread are allowed to execute at a time, even if the critical sections are different only one thread can be in one of them at a time.

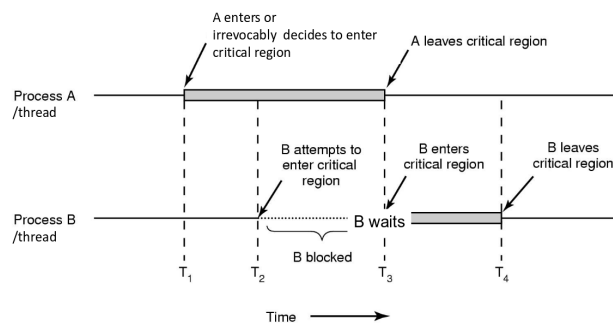


Figure 5.1: The desired outcome of the critical region problem.

We need to solve the problem of critical section in such a way that: only one thread at a time is allowed to execute a critical section; other threads can enter their critical sections in bounded time when no process is currently in its critical section; no thread will be starved, endlessly waiting for its turn of executing a critical section.

5.2.1 Check-competition approach

```
1. Shared var flagA, flagB: boolean; // indicate A's,
   B's intention to enter; initially both false
2. Thread A // B is symmetric
3.   repeat
4.     while flagB == true do [nothing] //busywait
5.     flagA  $\leftarrow$  true
6.     [critical section]
7.     flagA  $\leftarrow$  false
8.     [remainder section]
9.   forever
```

Figure 5.2: The check-competition approach to critical section problem.

One approach to the critical section problem is the Check-competition approach (see 5.2), this however fails at the mutual exclusion property, since several threads can manage to execute its line 4 before any thread manages to execute its line 5, blocking threads from executing line 4.

5.2.2 The check-order / after-you approach

```
1. Shared var turn: (idA..idB) // initially e.g. idA
   // indicates whose turn it is to enter CS
2. Thread A // B is symmetric
3.   repeat
4.     while turn  $\neq$  idA do [nothing] //busywait
5.     [critical section]
6.     turn  $\leftarrow$  idB
7.     [remainder section]
8.   forever
```

Figure 5.3: The check-order approach to critical section problem.

Another approach is called check order (see 5.3) and lets the threads take turn executing their critical section, instead waiting for their turn instead of deciding themselves when it's their time to execute. A problem with this however is starvation, since if the thread before never enters its critical section, all following threads will be stuck waiting forever.

5.2.3 Peterson's Algorithm

If we however combine the outcomes of these two approaches, we get a solution that works, Peterson's algorithm is this solution (see 5.4)

```
1. Shared var turn: (idA..idB) // initially e.g., idA
2. Shared var flagA, flagB: Boolean // initially false

3. thread A // B is symmetric
4.   repeat
5.     flagA ← true
6.     turn ← idB
7.     while (flagB and turn == idB) do [nothing]
8.     [critical section]
9.     flagA ← false
10.    [remainder section]
11.  forever
```

Figure 5.4: Peterson's algorithm, a working solution to the critical section problem.

5.3 Synchronization and hardware support

One hardware solution to the mutual exclusion problem is to just disable interrupts while in the critical section. There are however some problems with this.

- If we use a multiprocessor system, the threads running on different cores might still cause issues.
- If we run on a single-core system, nothing else can execute, meaning everything else will function.

5.3.1 Read-Modify-Write instructions

We can use an instruction called TestAndSet which atomically tests a value, and sets its value to true. Using this we can create locks which only one process are able to pass through.

```
TestAndSet (aka TAS) Instruction Definition:
boolean TestAndSet (boolean *target){
    boolean rv = *target;
    *target := true;
    return rv} // Executed atomically by the HW
```

Figure 5.5: Pseudo code for what Test and set does.

```
CompareAndSwap (aka CAS) Instruction Definition:
int CompareAndSwap(int *V, int expected, int
    new_value) {
    int temp := *V;
    if (temp == expected) then *V := new_value;
    return temp} // Executed atomically by the HW
```

Figure 5.6: Pseudo code for what compare and swap does.

5.3.2 Reflection on busy-waiting

Right now all the solutions are using busy-waiting, meaning they actively check for when they can continue. While this is a simple solution, it consumes computing resources without any reason to, it might starve, and deadlocks might happen if the thread waiting to use its critical section has higher priority than the one in a critical section, meaning the busy-waiting thread gets to execute its wait while the thread it waits for is not allowed to execute until the waiting thread has terminated.

5.4 Semaphores

5.5 Other techniques